The Cesame Programming Language

Project Report

Language Guru:	Qian Zhao (qz2512)
System Architect:	Teng Jiang (tj2488)
Tester:	Yunjia Wang (yw4105)

Contents

1 Introduction **3**

2 Language Tutorial: Cesame-101 **4**
2.1 Lexical Elements 4
2.2 Data Types 4
2.3 Expressions and Operators 4
2.4 Statements 4
2.5 Functions 5

3 Architectural Design **6**
3.1 File Organization 6
3.2 Components and Interfacing 6
3.3 Task Distribution 7

4 Test Plan **8**
4.1 Representative Source Language Programs: 8
4.2 Test Automation 14

5 Summary **15**
5.1 Individual Contributions 15
5.2 Takeaways from this Project 15
5.3 Advice for Future Teams 15

1 Introduction

Cesame is a C-like programming language that features native support for non-primitive types such as string, array, and structure. Our goal with Cesame is to create an imperative language that integrates strong typing, static typing, static scoping, and strict evaluation semantics. This means Cesame is designed to be both powerful and predictable in how it handles data and operations.

Behind the scenes, Cesame's compiler, written in Ocaml, translates high-level Cesame code into efficient lower-level code. By generating LLVM IR, an intermediate representation known for its versatility and optimization capabilities, Cesame's compiler strives to ensure that the resulting code achieves correctness and performance optimization to the best of its ability.

Our motivation for developing Cesame comes from a desire to build upon the strengths of C while addressing some of its limitations. We believe that by addressing shortcomings such as the lack of support for widely used data types like arrays, strings, and structures, we could not only improve our own proficiency as C programmers but also contribute to the wider programming community by making C more accessible and user-friendly for diverse applications. We call it "Cesame" because our goal is for this language to emulate the magical appeal of 'Open Sesame!!!', providing programmers with expanded capabilities within the C programming paradigm.

It's worth noting that Cesame extends beyond the confines of this course. We have plans to continue adding features and improvements even after the final submission. Our goal includes integrating first-class functions and further developing in Ocaml, leveraging the incorporation of C libraries. For the latest updates and contributions, we invite you to explore our GitHub repository: `https://github.com/cesame-pl/cesame`

2 Language Tutorial: Cesame-101

2.1 Lexical Elements

- **Identifiers**: Distinct names assigned to different elements within a program, adhering to specific naming conventions.
- **Keywords**: Words reserved by the language with predefined functionalities.
  (Refer to the Language Reference Manual for a comprehensive list.)
- **Separators**: Used to separate tokens such as white spaces or brackets.
- **Comments**: Denoted by `// single-line comments` or `/* general comments */`.

2.2 Data Types

- **Primitive Data Types**: `void`, `char`, `bool`, `int`, `float`
- **String**: a sequence of characters treated as a single data entity.
  - Declaring Strings: `String s;`
  - Defining Strings: `s = "Hello Cesame!"`
  - Declaring and defining Strings: `String n = "Cesame"`
- **Structures**: an unordered collection of different variables, including primitive types, String, structure and arrays.
  - Declaring Structures:

```
struct Student {
    String name;
    int age;
    Array<String> courses;
};
```

  - Defining and Declaring Variables of Structures:

```
Student a;
a.name = "Mike";
a.age = 22;
a.courses = ["math", "plt"];
```

- **Arrays**: containers that hold elements of the same type,
  - Declaring Arrays:

```
Array<int> a1;
Array<String> a2;
Array<char> a3;
Array<Array<int>> a4;
Array<Array<Array<int>>> a5
```

  - Defining, Accessing and Deleting Arrays:

```
Array<int> a = new [1, 2, 3];
Array<String> b = new ["Hello"];
println(b[0]);
delete a;
delete b;
```

2.3 Expressions and Operators

- Operators
  - Unary Operators: ++, –, -, !
  - Binary Operators: +, -, *, /, %
  - Comparison Operators: ==, !=, >=, <=, >, <
  - Logical Operators: &&, ||, !
  - Assignment Operators: =, +=, -=, *=, /=, %=
- Expressions: literals, variables, operators, function calls, etc.

2.4 Statements

- **Block**: a group of possibly empty statements enclosed in curly braces.
- **Expression Statements**: a statement that consists of an expression.
- **Declaration Statements**: introduces a new identifier.
- **Assignment Statements**: assigns a value to an already declared variable.

- **The `if` Statements**: flow control statement used for executing a block of code conditionally.
  - `if`:

    ```
    if (condition) {
        <stmts>
    }
    ```

  - `if-else`:

    ```
    if (condition) {
        <stmts>
    } else {
        <stmts>
    }
    ```

  - `if-elif-else`:

    ```
    if (condition) {
        <stmts>
    } elif {
        <stmts>
    } else {
        <stmts>
    }
    ```

- **The `while` Statements**:

```
while (condition) {
    <stmts>
    break;    // break is optional
    continue; // continue is optional
}
```

- **The `for` Statements**:

```
for (initialization; condition; step) {
    <stmts>
    break;    // break is optional
    continue; // continue is optional
}
```

2.5 Functions

- `Function Declaration and Definition`:

  Here's the basic structure of a function:

```
Func function_name(parameter_list)
        -> return_type {
    // Function body
    // Statements
    // Return statement (if applicable)
}
```

- `Declaration and Definition Example`:

  Here's a simple example of a function:

```
Func sum(int a, int b) -> int {
    int result = a + b;
    return result;
}
```

- `Calling Functions`:

  Here's how you call a function:

```
return_type result_variable =
    function_name(argument_list);
```

- `Calling Example`:

  Here's an example of calling the `sum` function:

```
int x = 5;
int y = 7;
int result = sum(x, y);
```

3 Architectural Design

3.1 File Organization

```
cesame ..... Root Directory of Project
├── src ..... Directory for Source Code
│   ├── scanner.mll ..... File for Scanner
│   ├── parser.mly ..... File for Parser
│   ├── ast.ml ..... File for AST Definition and Print
│   ├── sast.ml ..... File for SAST Definition and Print
│   ├── semant.ml ..... File for Semantic Check
│   ├── irgen.ml ..... File for IR Generation
│   ├── Makefile ..... Makefile for Source Code
│   │   ├── lib ..... Directory for C Library (Unused)
│   │   ├── printbig.c ..... File for C Library (Unused)
│   │   └── Makefile ..... Makefile for C Library (Unused)
├── test ..... Directory for Test Code
│   ├── output-a ..... Directory for AST Output
│   ├── output-s ..... Directory for SAST Output
│   └── output-l ..... Directory for LLVM Output
└── Makefile ..... Makefile for Making the Source and Generate Test Output
```

3.2 Components and Interfacing

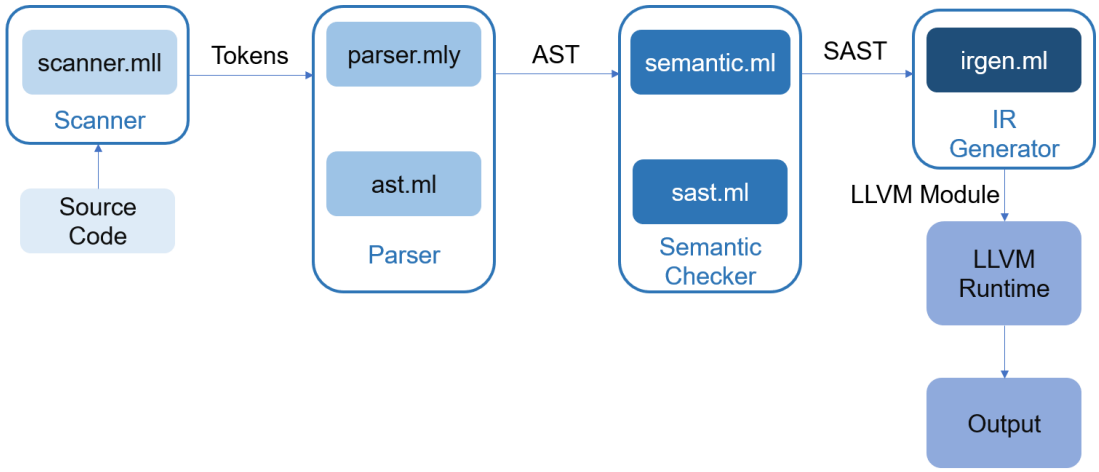


Figure 1: Block Diagram

As is shown in the file diagram and Fig.1, our code takes source code as input and generate LLVM intermediate representation (IR) as output. Then the Makefile will output the ASTs, SASTs and IRs of the source code in test directory into corresponding folder. If there is an error, the generation will generate the .err file and the generator will generate .output file is the compilation is successful at certain stage. We test the by using LLVM runtime to execute the IR. The processing of the source code is also shown in Fig.1.

The source code is first translated to tokens by scanner.mll and then passed to parser.mly. The parser will generate AST based on the context free grammar and AST definition in ast.ml.

The generated ast is passed to the semantic checker to generate the semantically checked AST. The semantic checker will check the semantics according to the SAST defined in sast.ml. The semantics includes type, valid operations on certain type, scope and visibility of variables and functions, meaning of assignments, correctness of branch sand loop statement, etc. And then SAST will be passed to IR generator to generate LLVM IR based on ocaml LLVM module.

3.3 Task Distribution

We work closely as a team and we have roughly same contributions. The project will not function without anyone of us.

- Qian: Array/String/Function Parsing, Rest of Semantic checker Control Flow IR, Makefile
- Teng: Declaration with initialization, Control Flow Parsing, Struct Parsing/Semantic Checking, Basic IR, Makefile
- Yunjia: Operations Parsing, Comments, Operations/Array/Function IR, Test case automation

4 Test Plan

4.1 Representative Source Language Programs:

- **Program 1: Koko eats banana**
  - Source Language Program

```
Func max_element(Array<int> arr, int n)->int {
   int max = arr[0];
   for (int i = 1; i < n; i++) {
      if (arr[i] > max) {
         max = arr[i];
      }
   }
   return max;
}
Func minEatingSpeed(Array<int> piles, int n, int h) ->int {
   int left = 1;
   int right = max_element(piles, n);
   while (left < right) {
      int mid = left + ((right - left) / 2);
      int hours = 0;
      for (int i = 0; i < n; i++) {
         hours += ((piles[i] + mid - 1) / mid);
      }
      if (hours <= h) {
         right = mid;
      } else {
         left = mid + 1;
      }
   }
   return left;
}
Array<int> piles = [3, 6, 7, 11];
println(minEatingSpeed(piles, 4, 8));
delete piles;
```

  - Parsed result

```
Parsed program:

int max_element(arr, n)
{
int max = arr[0];
for (int i = 1; i < n; i = i + 1) {
if (arr[i] > max)
{
max = arr[i];
}
}
return max;
}
int minEatingSpeed(piles, n, h)
{
int left = 1;
int right = max_element(piles, n);
while (left < right)
{
int mid = left + right - left / 2;
int hours = 0;
for (int i = 0; i < n; i = i + 1) {
hours = hours + piles[i] + mid - 1 / mid;
}
if (hours <= h)
{
right = mid;
}
else
{
left = mid + 1;
}
}
return left;
```

```
}
Array<int> piles = [3, 6, 7, 11];
println(minEatingSpeed(piles, 4, 8));
delete piles;
```

– Semantic checked result

```
Semantically checked program:

int max_element(arr, n)
{
int max = (int : (Array<int> : arr)[(int : 0)]);
for (int i = (int : 1); (bool : (int : i) < (int : n)); (int : (int : i) = (int : (int : i) + (int : 1)))){
if ((bool : (int : (Array<int> : arr)[(int : i)]) > (int : max)))
{
(int : (int : max) = (int : (Array<int> : arr)[(int : i)]));
}
}
return (int : max);
}
int minEatingSpeed(piles, n, h)
{
int left = (int : 1);
int right = (int : max_element((Array<int> : piles), (int : n)));
while ((bool : (int : left) < (int : right)))
{
int mid = (int : (int : left) + (int : (int : (int : right) - (int : left)) / (int : 2)));
int hours = (int : 0);
for (int i = (int : 0); (bool : (int : i) < (int : n)); (int : (int : i) = (int : (int : i) + (int : 1)))){
(int : (int : hours) = (int : (int : hours) + (int : (int : (int : (int : (Array<int> : piles)[(int : i)]) + (int : mid)) - (int : 1)) / (int :
      mid))));
}
if ((bool : (int : hours) <= (int : h)))
{
(int : (int : right) = (int : mid));
}
else
{
(int : (int : left) = (int : (int : mid) + (int : 1)));
}
}
return (int : left);
}
Array<int> piles = (Array<int> : [(int : 3), (int : 6), (int : 7), (int : 11)]);
(int : println((int : minEatingSpeed((Array<int> : piles), (int : 4), (int : 8)))));
delete (Array<int> : piles);
```

– LLVM IR

```
; ModuleID = 'cesame'
source_filename = "cesame"

@fmt = private unnamed_addr constant [4 x i8] c"%d\0A\00", align 1

declare i32 @printf(i8*, ...)

define i32 @main() {
entry:
  %piles = alloca i32*, align 8
  %malloccall = tail call i8* @malloc(i32 mul (i32 ptrtoint (i1** getelementptr (i1*, i1** null, i32 1) to i32), i32 4))
  %arr = bitcast i8* %malloccall to i32**
  %arr_ptr = bitcast i32** %arr to i32*
  %tmp = getelementptr inbounds i32, i32* %arr_ptr, i32 0
  store i32 3, i32* %tmp, align 4
  %tmp1 = getelementptr inbounds i32, i32* %arr_ptr, i32 1
  store i32 6, i32* %tmp1, align 4
  %tmp2 = getelementptr inbounds i32, i32* %arr_ptr, i32 2
  store i32 7, i32* %tmp2, align 4
  %tmp3 = getelementptr inbounds i32, i32* %arr_ptr, i32 3
  store i32 11, i32* %tmp3, align 4
  store i32* %arr_ptr, i32** %piles, align 8
  %piles4 = load i32*, i32** %piles, align 8
  %0 = call i32 @minEatingSpeed(i32* %piles4, i32 4, i32 8)
  %printf = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([4 x i8], [4 x i8]* @fmt, i32 0, i32 0), i32 %0)
```

```
  %piles5 = load i32*, i32** %piles, align 8
  %1 = bitcast i32* %piles5 to i8*
  tail call void @free(i8* %1)
  ret i32 0
}

define i32 @max_element(i32* %arr, i32 %n) {
entry:
  %arr1 = alloca i32*, align 8
  store i32* %arr, i32** %arr1, align 8
  %n2 = alloca i32, align 4
  store i32 %n, i32* %n2, align 4
  %max = alloca i32, align 4
  %tmp = load i32*, i32** %arr1, align 8
  %tmp3 = getelementptr inbounds i32, i32* %tmp, i32 0
  %tmp4 = load i32, i32* %tmp3, align 4
  store i32 %tmp4, i32* %max, align 4
  br label %init_for

init_for:                                         ; preds = %entry
  %i = alloca i32, align 4
  store i32 1, i32* %i, align 4
  br label %pred_for

pred_for:                                         ; preds = %trans_for, %init_for
  %i5 = load i32, i32* %i, align 4
  %n6 = load i32, i32* %n2, align 4
  %tmp7 = icmp slt i32 %i5, %n6
  br i1 %tmp7, label %while_body, label %merge

while_body:                                       ; preds = %pred_for
  %tmp8 = load i32*, i32** %arr1, align 8
  %i9 = load i32, i32* %i, align 4
  %tmp10 = getelementptr inbounds i32, i32* %tmp8, i32 %i9
  %tmp11 = load i32, i32* %tmp10, align 4
  %max12 = load i32, i32* %max, align 4
  %tmp13 = icmp sgt i32 %tmp11, %max12
  br i1 %tmp13, label %then, label %else

merge:                                            ; preds = %pred_for
  %max21 = load i32, i32* %max, align 4
  ret i32 %max21

trans_for:                                        ; preds = %merge14
  %i19 = load i32, i32* %i, align 4
  %tmp20 = add i32 %i19, 1
  store i32 %tmp20, i32* %i, align 4
  br label %pred_for

merge14:                                          ; preds = %else, %then
  br label %trans_for

then:                                             ; preds = %while_body
  %tmp15 = load i32*, i32** %arr1, align 8
  %i16 = load i32, i32* %i, align 4
  %tmp17 = getelementptr inbounds i32, i32* %tmp15, i32 %i16
  %tmp18 = load i32, i32* %tmp17, align 4
  store i32 %tmp18, i32* %max, align 4
  br label %merge14

else:                                             ; preds = %while_body
  br label %merge14
}

define i32 @minEatingSpeed(i32* %piles, i32 %n, i32 %h) {
entry:
  %piles1 = alloca i32*, align 8
  store i32* %piles, i32** %piles1, align 8
  %n2 = alloca i32, align 4
  store i32 %n, i32* %n2, align 4
  %h3 = alloca i32, align 4
  store i32 %h, i32* %h3, align 4
  %left = alloca i32, align 4
  store i32 1, i32* %left, align 4
  %right = alloca i32, align 4
  %n4 = load i32, i32* %n2, align 4
```

```
  %piles5 = load i32*, i32** %piles1, align 8
  %0 = call i32 @max_element(i32* %piles5, i32 %n4)
  store i32 %0, i32* %right, align 4
  br label %while

while:                                            ; preds = %merge35, %entry
  %left6 = load i32, i32* %left, align 4
  %right7 = load i32, i32* %right, align 4
  %tmp = icmp slt i32 %left6, %right7
  br i1 %tmp, label %while_body, label %merge

while_body:                                       ; preds = %while
  %mid = alloca i32, align 4
  %left8 = load i32, i32* %left, align 4
  %right9 = load i32, i32* %right, align 4
  %left10 = load i32, i32* %left, align 4
  %tmp11 = sub i32 %right9, %left10
  %tmp12 = sdiv i32 %tmp11, 2
  %tmp13 = add i32 %left8, %tmp12
  store i32 %tmp13, i32* %mid, align 4
  %hours = alloca i32, align 4
  store i32 0, i32* %hours, align 4
  br label %init_for

merge:                                            ; preds = %while
  %left39 = load i32, i32* %left, align 4
  ret i32 %left39

init_for:                                         ; preds = %while_body
  %i = alloca i32, align 4
  store i32 0, i32* %i, align 4
  br label %pred_for

pred_for:                                         ; preds = %trans_for, %init_for
  %i15 = load i32, i32* %i, align 4
  %n16 = load i32, i32* %n2, align 4
  %tmp17 = icmp slt i32 %i15, %n16
  br i1 %tmp17, label %while_body14, label %merge18

while_body14:                                     ; preds = %pred_for
  %hours19 = load i32, i32* %hours, align 4
  %tmp20 = load i32*, i32** %piles1, align 8
  %i21 = load i32, i32* %i, align 4
  %tmp22 = getelementptr inbounds i32, i32* %tmp20, i32 %i21
  %tmp23 = load i32, i32* %tmp22, align 4
  %mid24 = load i32, i32* %mid, align 4
  %tmp25 = add i32 %tmp23, %mid24
  %tmp26 = sub i32 %tmp25, 1
  %mid27 = load i32, i32* %mid, align 4
  %tmp28 = sdiv i32 %tmp26, %mid27
  %tmp29 = add i32 %hours19, %tmp28
  store i32 %tmp29, i32* %hours, align 4
  br label %trans_for

merge18:                                          ; preds = %pred_for
  %hours32 = load i32, i32* %hours, align 4
  %h33 = load i32, i32* %h3, align 4
  %tmp34 = icmp sle i32 %hours32, %h33
  br i1 %tmp34, label %then, label %else

trans_for:                                        ; preds = %while_body14
  %i30 = load i32, i32* %i, align 4
  %tmp31 = add i32 %i30, 1
  store i32 %tmp31, i32* %i, align 4
  br label %pred_for

merge35:                                          ; preds = %else, %then
  br label %while

then:                                             ; preds = %merge18
  %mid36 = load i32, i32* %mid, align 4
  store i32 %mid36, i32* %right, align 4
  br label %merge35

else:                                             ; preds = %merge18
  %mid37 = load i32, i32* %mid, align 4
```

```
  %tmp38 = add i32 %mid37, 1
  store i32 %tmp38, i32* %left, align 4
  br label %merge35
}

declare noalias i8* @malloc(i32)

declare void @free(i8*)
```

– Output

```
4
```

- **Program 2**
  - Source Language Program

```
struct Student {
   String name;
   int age;
   Array<String> courses;
};

Student a;
a.name = "Mike";
a.age = 22;
a.courses = ["math", "history", "cs"];
a.courses[0] = "french";

print("Mike's courses: ");
for (int i = 0; i < 3; i++) {
   print(a.courses[i]);
   if (i!= 2) {
      print(", ");
   }
}
```

  - Parsed result

```
Parsed program:

struct Student
{
String name;
int age;
Array<String> courses;
}
Student a;
a.name = "Mike";
a.age = 22;
a.courses = ["math", "history", "cs"];
a.courses[0] = "french";
print("Mike's courses: ");
for (int i = 0; i < 3; i = i + 1) {
print(a.courses[i]);
if (i != 2)
{
print(", ");
}
}
```

  - Semantic checked result

```
Semantically checked program:

struct Student
{
String name;
int age;
Array<String> courses;
}
```

```
Student a;
(String : (String : (Student : a).(String : name)) = (String : "Mike"));
(int : (int : (Student : a).(int : age)) = (int : 22));
(Array<String> : (Array<String> : (Student : a).(Array<String> : courses)) = (Array<String> : [(String : "math"), (String : "history"), (String : "cs")]));
(String : (String : (Array<String> : (Student : a).(Array<String> : courses))[(int : 0)]) = (String : "french"));
(int : print((String : "Mike's courses: ")));
for (int i = (int : 0); (bool : (int : i) < (int : 3)); (int : (int : i) = (int : (int : i) + (int : 1)))){
(int : print((String : (Array<String> : (Student : a).(Array<String> : courses))[(int : i)])));
if ((bool : (int : i) != (int : 2)))
{
(int : print((String : ", ")));
}
}
```

– LLVM IR

```
; ModuleID = 'cesame'
source_filename = "cesame"

@Student = global { i8*, i32, i8** } zeroinitializer
@str = private unnamed_addr constant [5 x i8] c"Mike\00", align 1
@str.1 = private unnamed_addr constant [5 x i8] c"math\00", align 1
@str.2 = private unnamed_addr constant [8 x i8] c"history\00", align 1
@str.3 = private unnamed_addr constant [3 x i8] c"cs\00", align 1
@str.4 = private unnamed_addr constant [7 x i8] c"french\00", align 1
@fmt = private unnamed_addr constant [3 x i8] c"%s\00", align 1
@str.5 = private unnamed_addr constant [17 x i8] c"Mike's courses: \00", align 1
@fmt.6 = private unnamed_addr constant [3 x i8] c"%s\00", align 1
@fmt.7 = private unnamed_addr constant [3 x i8] c"%s\00", align 1
@str.8 = private unnamed_addr constant [3 x i8] c", \00", align 1

declare i32 @printf(i8*, ...)

define i32 @main() {
entry:
  %a = alloca { i8*, i32, i8** }, align 8
  %tmp = getelementptr inbounds { i8*, i32, i8** }, { i8*, i32, i8** }* %a, i32 0, i32 0
  store i8* getelementptr inbounds ([5 x i8], [5 x i8]* @str, i32 0, i32 0), i8** %tmp, align 8
  %tmp1 = getelementptr inbounds { i8*, i32, i8** }, { i8*, i32, i8** }* %a, i32 0, i32 1
  store i32 22, i32* %tmp1, align 4
  %tmp2 = getelementptr inbounds { i8*, i32, i8** }, { i8*, i32, i8** }* %a, i32 0, i32 2
  %malloccall = tail call i8* @malloc(i32 mul (i32 ptrtoint (i1** getelementptr (i1*, i1** null, i32 1) to i32), i32 3))
  %arr = bitcast i8* %malloccall to i8***
  %arr_ptr = bitcast i8*** %arr to i8**
  %tmp3 = getelementptr inbounds i8*, i8** %arr_ptr, i32 0
  store i8* getelementptr inbounds ([5 x i8], [5 x i8]* @str.1, i32 0, i32 0), i8** %tmp3, align 8
  %tmp4 = getelementptr inbounds i8*, i8** %arr_ptr, i32 1
  store i8* getelementptr inbounds ([8 x i8], [8 x i8]* @str.2, i32 0, i32 0), i8** %tmp4, align 8
  %tmp5 = getelementptr inbounds i8*, i8** %arr_ptr, i32 2
  store i8* getelementptr inbounds ([3 x i8], [3 x i8]* @str.3, i32 0, i32 0), i8** %tmp5, align 8
  store i8** %arr_ptr, i8*** %tmp2, align 8
  %tmp6 = getelementptr inbounds { i8*, i32, i8** }, { i8*, i32, i8** }* %a, i32 0, i32 2
  %tmp7 = load i8**, i8*** %tmp6, align 8
  %tmp8 = getelementptr inbounds i8*, i8** %tmp7, i32 0
  store i8* getelementptr inbounds ([7 x i8], [7 x i8]* @str.4, i32 0, i32 0), i8** %tmp8, align 8
  %printf = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @fmt, i32 0, i32 0), i8* getelementptr inbounds ([17 x i8], [17 x i8]* @str.5, i32 0, i32 0))
  br label %init_for

init_for:                                         ; preds = %entry
  %i = alloca i32, align 4
  store i32 0, i32* %i, align 4
  br label %pred_for

pred_for:                                         ; preds = %trans_for, %init_for
  %i9 = load i32, i32* %i, align 4
  %tmp10 = icmp slt i32 %i9, 3
  br i1 %tmp10, label %while_body, label %merge

while_body:                                       ; preds = %pred_for
  %tmp11 = getelementptr inbounds { i8*, i32, i8** }, { i8*, i32, i8** }* %a, i32 0, i32 2
  %tmp12 = load i8**, i8*** %tmp11, align 8
  %i13 = load i32, i32* %i, align 4
  %tmp14 = getelementptr inbounds i8*, i8** %tmp12, i32 %i13
```

```
  %tmp15 = load i8*, i8** %tmp14, align 8
  %printf16 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @fmt.6, i32 0, i32 0), i8* %tmp15)
  %i17 = load i32, i32* %i, align 4
  %tmp18 = icmp ne i32 %i17, 2
  br i1 %tmp18, label %then, label %else

merge:                                            ; preds = %pred_for
  ret i32 0

trans_for:                                        ; preds = %merge19
  %i21 = load i32, i32* %i, align 4
  %tmp22 = add i32 %i21, 1
  store i32 %tmp22, i32* %i, align 4
  br label %pred_for

merge19:                                          ; preds = %else, %then
  br label %trans_for

then:                                             ; preds = %while_body
  %printf20 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds ([3 x i8], [3 x i8]* @fmt.7, i32 0, i32 0), i8* getelementptr inbounds ([3
      x i8], [3 x i8]* @str.8, i32 0, i32 0))
  br label %merge19

else:                                             ; preds = %while_body
  br label %merge19
}

declare noalias i8* @malloc(i32)
```

– output

```
Mike's courses: french, history, cs
```

4.2 Test Automation

- Our testing process is extensively automated and integrated into the Makefile. It performs the following tasks:
  - Cleans previous build artifacts.
  - Compiles source files and required libraries to build the Cesame executable.
  - Runs Cesame executable with various flags (-a, -s, -l) on all files in the test directory, saving output or error files in designated locations.
  - Executes programs in LLVM bitcode format and stores the results in the test directory using the command `lli <llvm_bitcode.out>`.
  - A summary of the tests is shown.
- To initiate the test automation process, simply navigate to the Cesame directory and run `make`

5 Summary

5.1 Individual Contributions

Qian Zhao:

- Makefile: The initial Makefile for semantic checker
- Parsing: Handled string and array
- Semantics: Completed scope check, array type check, for, while, if, break, continue semantic check, and basic function declaration
- IR generation: IR generation for for while if break and continue.
- Testing: Created test files for scope checking.

Yunjia Wang:

- Makefile: Incorporated LLVM packages and automated testing for all files in the test folder.
- Parsing: Handled comments, operators, floats, and characters.
- Semantics: Completed the implementation for accessing array elements.
- IR generation: Finished LLVM IR code for operators, arrays, structs, functions, and printing.
- Testing: Created test files for both failed and valid cases.

Teng Jiang:

- LRM: language principles design, especially for Struct, Array and Func.
- Parsing: Conditions, control flows.
- Semantics: Struct semantic checking.
- IR generation: Scheleton code for IR generations. (Basic building blocks and helper functions)
- Testing: Created test files for both failed and valid cases.

5.2 Takeaways from this Project

Qian Zhao:

- Achieved a deeper understanding for scoping and type checking
- Understand the fundamental execution logic of C-like languages.

Yunjia Wang:

- Achieved a comprehensive understanding of parsing and semantic analysis.
- Improved proficiency in generating LLVM IR code.

Teng Jiang:

- Object oriented programming provides flexibility for the programmers, but it's really trick to handle as a language developer.
- A good testing workflow is the key to success.

5.3 Advice for Future Teams

Qian Zhao:

- Consider the IR generation when writing semantic checker (before actually doing the IR).

Yunjia Wang:

- Make sure everyone knows what we're aiming for and where they fit in.
- Things change, plans shift, and that's okay! Stay nimble, stay flexible, and we'll tackle whatever curveballs come our way with a smile.

Teng Jiang:

- Start early.
- Start testing early!!!